BÖLÜM 6 / MODULE 6

Dağıtık Sistem Temelleri ve Eşgüdüm

Distributed Systems Fundamentals, Consensus & Coordination

Saat kaymaları ve Vektör Saatleri, Raft fikir birliği algoritması, etcd/ZooKeeper kilitleri, Twitter Snowflake ID üreteci, Saga deseni ve Circuit Breaker.

Clock drift & Vector Clocks, Raft consensus mechanics, etcd/ZooKeeper coordination, Twitter Snowflake ID generators, Saga vs 2PC, and Circuit Breaker resilience.

31

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Saatler ve Zaman Sorunları: Saat Kayması (Clock Drift), NTP ve Vektör Saatleri

Distributed Clocks & Time: Physical Drift, NTP & Vector Clocks

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ FİZİKSEL SAAT KAYMASI (CLOCK DRIFT & NTP SKEW) ]
Sunucu A Saati: [ 12:00:00.100 ] ──► Olay X (Gerçekte önce oldu!)
Sunucu B Saati: [ 12:00:00.050 ] ──► Olay Y (Saat geri kaldığı için önce gibi görünüyor!)
(Last-Write-Wins uygularsanız Olay X haksız yere ezilir ve silinir!)

[ MANTIKSAL VEKTÖR SAATI (VECTOR CLOCK) ]
Her düğüm kendi sayacını tutar: V = [NodeA: 1, NodeB: 0, NodeC: 2]
Eğer V1'deki tüm elemanlar <= V2 ise: V1 kesinlikle V2'den ÖNCE olmuştur (Happens-Before)!
Eğer çelişki varsa (V1 > V2 ve V2 > V1): Eşzamanlı çakışma (Concurrent Conflict) tespit edilir!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kolunuzdaki mekanik saatin her gün 2 saniye ileri gittiğini, arkadaşınızın saatinin ise 3 saniye geri kaldığını düşünün. İkiniz de 'Ben mesajı önce attım' diye kavga ederseniz kol saatlerinize bakarak kimin haklı olduğunu ASLA çözemezsiniz! Bilgisayarlardaki kuvars saatler de sıcaklıktan ve elektrikten dolayı kayar (Clock Drift). Dağıtık sistemlerde 'fiziksel duvardaki saate' güvenemezsiniz! Bunun yerine basketbol maçındaki paslaşma sırası gibi çalışan 'Mantıksal Saatler' (Lamport / Vector Clocks) kullanılır: 'Top önce bendeydi, ben sana attım, sonra sen kaleye attın; saatin kaç olduğu kimin umurunda!'

🇬🇧 ENGLISH (ELI5)

Imagine your wristwatch gains 2 seconds every day, while your friend's wristwatch loses 3 seconds. If you both send letters claiming 'I mailed mine first', comparing your wristwatches is completely useless! Computer quartz crystals drift due to thermal fluctuations (Clock Drift). In distributed systems, relying on physical wall-clock time (`System.currentTimeMillis()`) is dangerous! Instead, systems use Logical Clocks (Lamport & Vector Clocks): 'I held the ball first, passed it to you, and then you scored; physical clock ticks are irrelevant, causal sequence is everything!'

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık sistemlerin en büyük yanılgısı: 'Fiziksel saatler eşzamanlıdır'. NTP (Network Time Protocol) saatleri senkronize etmeye çalışır ancak ağ dalgalanmaları nedeniyle sunucular arasında 10ms - 100ms arasında saat kayması (Clock Skew) kaçınılmazdır. 'Last-Write-Wins' (LWW) fiziksel saate dayanırsa veri kaybına yol açar (Google Spanner bunu çözmek için atomik saatler ve GPS alıcıları kullanan TrueTime API ile belirsizlik aralığını [ε ≈ 7ms] sınırlandırmıştır). Leslie Lamport'un Mantıksal Saatleri ve Vektör Saatleri (Vector Clocks), fiziksel saate ihtiyaç duymadan 'Happens-Before' nedensellik ilişkisini (A → B) matematiksel olarak modeller ve eşzamanlı çakışmaları (Concurrent Updates) tespit eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The fallacy of physical time in distributed nodes: NTP continuously steers hardware crystals, but asymmetric network jitter introduces 10ms-100ms clock skew. Naive Last-Write-Wins (LWW) relying on wall-clock time causes silent data loss. Google Spanner tackled this by placing GPS receivers and atomic clocks in every datacenter, bounding physical drift uncertainty to a narrow window (ε ≈ 7ms via TrueTime). Leslie Lamport proved that causality (A → B) can be modeled logically without physical clocks. Vector Clocks track an array of monotonic counters across nodes: if neither clock dominates the other, a concurrent conflict is flagged and handed to application logic for resolution.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Vektör Saati (Vector Clock) ile Nedensellik ve Çakışma Tespiti
import java.util.*;

public class VectorClock {
    private final Map<String, Integer> clock = new HashMap<>();

    public VectorClock() {}
    public VectorClock(Map<String, Integer> copy) { clock.putAll(copy); }

    public synchronized void tick(String nodeId) {
        clock.put(nodeId, clock.getOrDefault(nodeId, 0) + 1);
    }

    // İki vektör saatinin nedensellik durumunu karşılaştır
    public enum Relation { BEFORE, AFTER, CONCURRENT }

    public static Relation compare(VectorClock v1, VectorClock v2) {
        boolean v1Greater = false;
        boolean v2Greater = false;

        Set<String> allNodes = new HashSet<>(v1.clock.keySet());
        allNodes.addAll(v2.clock.keySet());

        for (String node : allNodes) {
            int c1 = v1.clock.getOrDefault(node, 0);
            int c2 = v2.clock.getOrDefault(node, 0);
            if (c1 > c2) v1Greater = true;
            if (c2 > c1) v2Greater = true;
        }

        if (v1Greater && !v2Greater) return Relation.AFTER;
        if (!v1Greater && v2Greater) return Relation.BEFORE;
        return Relation.CONCURRENT; // Çelişki / Eşzamanlı Çakışma!
    }
}

void main() {
    var v1 = new VectorClock();
    v1.tick("NodeA"); // [A:1]

    var v2 = new VectorClock(v1.clock);
    v2.tick("NodeB"); // [A:1, B:1] -> v2, v1'den sonra olmuştur!

    IO.println("V1 ve V2 İlişkisi: " + VectorClock.compare(v1, v2)); // BEFORE
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta dağıtık bir sistemde iki olayın hangisinin önce gerçekleştiğini belirlemek için `System.currentTimeMillis()` kullanacağınızı ASLA söylemeyin! NTP saati geriye atlayabilir (Leap Second veya NTP sync). Her zaman Mantıksal Saat veya Monotonic Sequence Number önerin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose sorting distributed transactions across multiple servers using `System.currentTimeMillis()`. NTP adjustments can cause system clocks to tick backwards! Always prescribe Logical Clocks, Lamport timestamps, or monotonic sequence counters.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Fiziksel saatler donanım kayması ve NTP gecikmesi nedeniyle güvenilmezdir.

• Last-Write-Wins (LWW) saati geri kalan sunucudaki güncel verilerin silinmesine yol açabilir.

• Vektör saatleri (Vector Clocks) fiziksel zamana ihtiyaç duymadan çakışmaları tespit eder.

🎯 Key Takeaways for the Interview

• Physical clocks drift constantly due to hardware temperature and NTP transit jitter.

• Last-Write-Wins risks silently dropping valid writes if clock skew is present.

• Vector Clocks accurately trace causal order and isolate concurrent update conflicts.

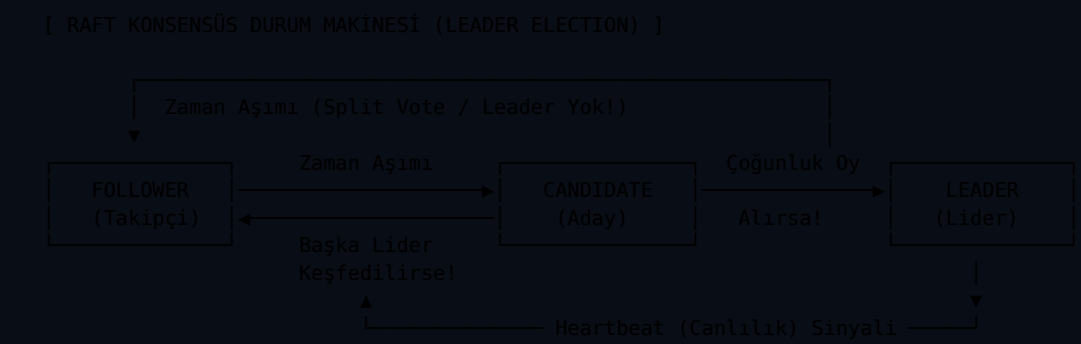
32

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Fikir Birliği (Consensus): Raft Algoritması (Lider Seçimi ve Replikasyon)

Distributed Consensus: The Raft Algorithm (Leader Election & Logs)

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

5 kişilik bir arkadaş grubunun tatile nereye gideceğine karar vermesini düşünün. Herkes aynı anda konuşursa kaos çıkar. Raft algoritması şudur: 1) İçlerinden biri 'Ben grup lideri olmak istiyorum, bana oy verin' der (Candidate). 2) Eğer arkadaşlarının çoğunluğu (en az 3 kişi) 'Tamam, sen lidersin' derse o kişi Lider olur. 3) Artık tüm kararları lider alır, gruba söyler ve çoğunluk onaylayınca deftere yazar. Lider aniden bayılırsa (Crash), arkadaşlarından biri 'Liderden 150 milisaniyedir ses çıkmıyor' diyerek yeni bir seçim başlatır!

🇬🇧 ENGLISH (ELI5)

Imagine 5 friends voting on a vacation spot. If everyone screams conflicting orders, total chaos erupts. Raft brings democratic order: 1) A member proposes: 'I volunteer as Leader, vote for me!' (Candidate). 2) If a majority quorum (at least 3 out of 5) grants their vote, they become the undisputed Leader. 3) The Leader proposes decisions, replicates the entry to nodes, and commits once a majority confirms. If the Leader faints (crashes), an election timer expires and nodes elect a fresh leader within milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Raft (Ongaro & Ousterhout, Stanford 2014), anlaşılması imkansız derecede karmaşık olan Paxos algoritmasının yerine geliştirilmiş, anlaşılabilir bir konsensüs algoritmasıdır (etcd, Kubernetes, Kafka KRaft, TiDB tarafından kullanılır). 3 temel alt problemden oluşur: 1) Lider Seçimi (Leader Election): Liderden belirli bir süre (Heartbeat timeout: 150-300ms) kalp atışı alamayan Follower, rastgele bir seçim zaman aşımıyla (Randomized Election Timeout) Candidate olur ve `RequestVote` fırlatır. Çoğunluk oyu alan ($N/2 + 1$) lider seçilir. 2) Kütük Replikasyonu (Log Replication): Lider istemciden komutu alır, kendi kütüğüne yazar, follower'lara `AppendEntries` gönderir. Düğümlerin çoğunluğu diske yazdığında işlem 'Committed' kabul edilir. 3) Güvenlik (Safety): Yeni seçilen lider, kütüğü en güncel olan düğüm olmak zorundadır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Raft decomposes consensus into three decomposed state machines: 1) Leader Election: Followers transition to Candidates when heartbeat deadlines (150-300ms randomized timer) expire without leader contact. A candidate requests votes; securing a strict majority quorum ($(N/2) + 1$) crowns the Leader. 2) Log Replication: The Leader appends entries locally, broadcasting `AppendEntries` RPCs. When an entry is replicated across a majority, the Leader commits it and applies to the state machine. 3) Election Safety: A follower will deny vote if the candidate's log is less up-to-date than its own, guaranteeing committed entries are never overwritten or lost. Powers etcd, Kubernetes control planes, and Kafka KRaft.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Raft Rastgele Seçim Zaman Aşımı (Randomized Election Timeout)
import java.util.concurrent.*;

public class RaftElectionTimer {
    public enum Role { FOLLOWER, CANDIDATE, LEADER }

    private Role currentRole = Role.FOLLOWER;
    private long currentTerm = 0;

    public synchronized void onHeartbeatTimeout() {
        if (currentRole != Role.LEADER) {
            currentRole = Role.CANDIDATE;
            currentTerm++;
            int randomTimeout = ThreadLocalRandom.current().nextInt(150, 300);
            IO.println("Term #" + currentTerm + ": Lider düştü! Aday olundu (Timeout: " +
randomTimeout + "ms)");
        }
    }

    public synchronized void becomeLeader() {
        currentRole = Role.LEADER;
        IO.println("🏆 DÜĞÜM LİDER SEÇİLDİ! Term #" + currentTerm);
    }
}

void main() {
    var node = new RaftElectionTimer();
    node.onHeartbeatTimeout();
    node.becomeLeader();
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Raft kümesindeki düğüm sayısı daima TEK SAYI (Odd Number: 3, 5 veya 7) olmalıdır! 4 düğümlü bir kümede 2'ye 2 bölünme (Split Vote) yaşanır ve çoğunluk ($4/2 + 1 = 3$) sağlanamayabilir. 3 düğüm 1 hatayı, 5 düğüm 2 hatayı tolere eder.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Always size Raft clusters with an ODD number of nodes (3, 5, or 7). An even count (4 nodes) offers zero additional fault tolerance over 3 nodes (both tolerate only 1 failure) while elevating split-vote deadlocks.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Raft, dağıtık sistemlerde lider seçimi ve veri replikasyonunu güvenceye alır.
- Rastgele seçim zaman aşımları (Randomized Timeouts) oyların bölünmesini (Split Vote) önler.
- Kubernetes (etcd) ve Kafka (KRaft) fikir birliği omurgasını Raft üzerine kurmuştur.

🎯 Key Takeaways for the Interview

- Raft guarantees leader election and deterministic log replication across distributed nodes.
- Randomized election timeouts disperse candidacy announcements, avoiding voting ties.
- Kubernetes (via etcd) and Kafka KRaft rely directly on Raft for operational consensus.

33

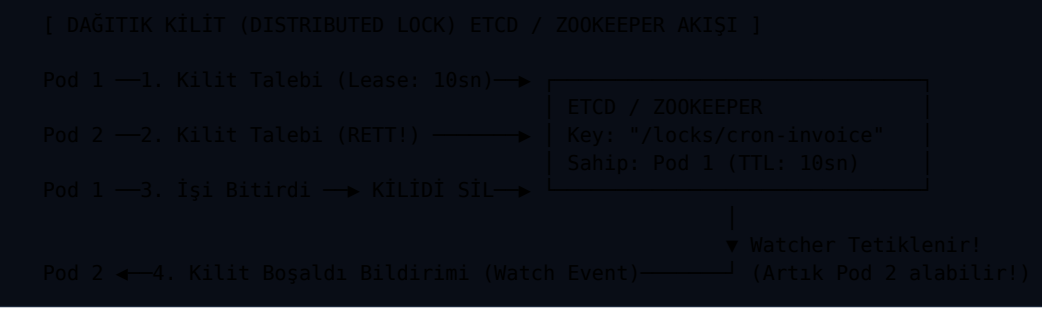
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Eşgüdüm Servisi: Apache ZooKeeper ve etcd (Dağıtık Kilit & Lider)

Distributed Coordination: ZooKeeper & etcd (Distributed Locks & Leader)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

10 odalı bir pansiyonda tek bir ortak banyo olduğunu düşünün. Banyonun kapısına 'Dolu / Boş' yazan bir kilit asarsınız (Dağıtık Kilit - Distributed Lock). İçeri giren kişi kapıyı kilitler; dışarıdakiler kapının açılmasını bekler (Watcher). Eğer içerideki kişi banyoda bayılırsa kapı sonsuza kadar kilitli kalmasın diye kilit 10 dakika sonra otomatik açılır (Lease / TTL). İşte `etcd` ve `ZooKeeper`, binlerce sunucunun aynı anda tek bir kaynağa hücum edip birbirini ezmesini engelleyen bu güvenilir kilit servisidir!

🇬🇧 ENGLISH (ELI5)

Picture a boarding house with 10 rooms and a single shared bathroom. On the door hangs a physical latch (Distributed Lock). When someone enters, they slide the latch; others queue outside watching the door (Watcher). To prevent the door locking forever if someone falls asleep inside, the latch springs open automatically after 10 minutes (Lease / TTL). `etcd` and `ZooKeeper` serve as that centralized, bulletproof lock coordinator for thousands of server pods!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

etcd ve Apache ZooKeeper; hiyerarşik anahtar-değer ağacı (znodes), güçlü tutarlılık (Raft / Zab), geçici düğümler (Ephemeral nodes / Leases) ve olay dinleyicileri (Watchers) sunan dağıtık eşgüdüm servisleridir. Kullanım Alanları: 1) Dağıtık Kilit (Distributed Lock): Tek bir cron görevinin sadece tek bir podda çalışmasını sağlamak. 2) Lider Seçimi (Leader Election): Kümedeki podlardan biri geçici bir düğüm (`/leader`) oluşturur; çökerse Lease düşer ve Watcher diğer podları uyandırarak yeni lideri seçtirir. 3) Konfigürasyon Yönetimi: Sistem konfigürasyonunu merkezi olarak tutar; konfigürasyon değiştiğinde tüm mikroservisler anında Watcher ile güncellenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

etcd and ZooKeeper provide strongly consistent, linearizable key-value hierarchies backed by Raft/ZAB consensus. Core primitives: 1) Distributed Locking: A worker acquires a key bound to a time-limited Lease (heartbeat keep-alive). If the worker crashes, the lease expires and purges the key automatically, avoiding deadlocks. 2) Leader Election: Nodes race to claim an ephemeral key (`/leader/worker-A`). The winner leads; competitors register Watchers on the key path to react instantly to failures. 3) Dynamic Configuration: Centralizes runtime flags; changes trigger push notifications across thousands of microservice Watcher streams within milliseconds.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Dağıtık Kilit (Distributed Lock) Mantığı Simülasyonu
import java.util.concurrent.*;

public class DistributedLockSim {
    private String lockOwner = null;
    private long leaseExpiration = 0;

    public synchronized boolean acquireLock(String workerId, long leaseDurationMillis) {
        long now = System.currentTimeMillis();
        // Kilit boş mu veya süresi dolmuş mu?
        if (lockOwner == null || now > leaseExpiration) {
            lockOwner = workerId;
            leaseExpiration = now + leaseDurationMillis;
            IO.println("🔒 Kilit Alındı: " + workerId + " (Süre: " + leaseDurationMillis + "ms)");
            return true;
        }
        return false; // Kilit meşgul!
    }

    public synchronized void releaseLock(String workerId) {
        if (workerId.equals(lockOwner)) {
            lockOwner = null;
            IO.println("🔓 Kilit Bırakıldı: " + workerId);
        }
    }
}

void main() {
    var locker = new DistributedLockSim();
    boolean pod1 = locker.acquireLock("Pod-1", 5000);
    boolean pod2 = locker.acquireLock("Pod-2", 5000); // Başarısız olmalı!

    IO.println("Pod-1 Başarılı mı? " + pod1);
    IO.println("Pod-2 Başarılı mı? " + pod2);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

etcd veya ZooKeeper'ı devasa veriler (büyük JSON'lar, resimler) saklamak için KULLANMAYIN! Bu sistemlerin bellek sınırı ve mimarisi yalnızca küçük metadata ve kilitler (birkaç kilobayt) içindir; büyük veriler Raft konsensüs kütüğünü kilitler.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never store bulky payloads inside etcd or ZooKeeper. They are strictly designed for micro-metadata and consensus coordination (under 1MB per value). Storing heavy state degrades Raft commit replication across nodes.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- etcd ve ZooKeeper, dağıtık sistemlerde kilit ve lider seçiminin güvenilir adresidir.
- Lease (Kira / TTL) mekanizması, düğüm çöktüğünde kilitlerin sonsuza kadar kilitli kalmasını (Deadlock) önler.
- Watcher API, veritabanını sürekli sorgulamak (polling) yerine olay tabanlı anlık bildirim sağlar.

🎯 Key Takeaways for the Interview

- etcd and ZooKeeper serve as the authoritative standard for distributed locks and elections.
- Time-bound leases prevent distributed deadlocks by releasing locks upon node failure.
- Watcher streams replace expensive database polling with event-driven notifications.

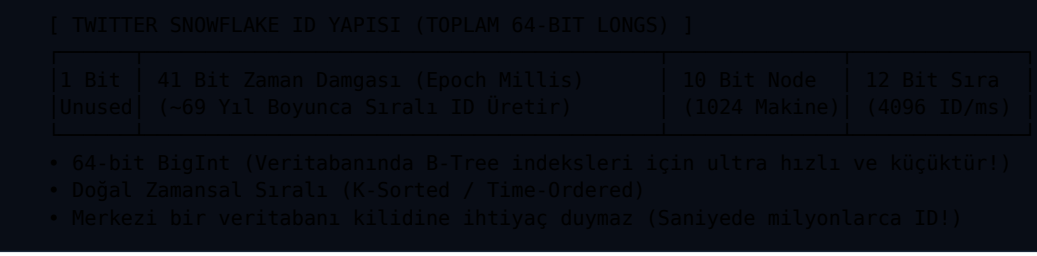
34

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Benzersiz ID Üretimi: Twitter Snowflake, UUIDv7 ve Ticket Server

Distributed Unique ID Generation: Twitter Snowflake & UUIDv7

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir doğum hastanesini düşünün: Her doğan bebeğe benzersiz bir kimlik numarası verilmelidir. Eskiden tek bir memur sırayla 1, 2, 3… diye numara verirdi (Auto-Increment). Ama dünyada aynı anda 100 hastanede bebek doğuyorsa hepsi Ankara'daki o tek memura telefon açamaz (Darboğaz ve çöküş!). Twitter Snowflake ise her hastaneye özel bir formül verir: 'Önce şu anki saati yaz, yanına kendi hastane kodunu ekle, yanına da o saniyedeki bebek sırasını ekle!' Böylece hiçbir hastane birbirine telefon etmeden saniyede milyonlarca benzersiz ve kronolojik olarak sıralı kimlik numarası üretir!

🇬🇧 ENGLISH (ELI5)

Imagine assigning national birth certificate numbers. A naive approach is a single bureaucrat sitting in the capital stamping 1, 2, 3… sequentially (Auto-Increment ID). But if 1,000 maternity hospitals across the nation deliver babies simultaneously, that single bureaucrat causes nationwide traffic jams! Twitter Snowflake equips every hospital with a decentralized formula: 'Stamp the current millisecond timestamp, append your hospital ID, append a millisecond sequence counter!' No hospital coordinates with another, yet every generated ID is universally unique and strictly chronological!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık mimarilerde ID gereksinimleri: Benzersizlik, zamana göre sıralanabilirlik (K-Sortable / B-Tree indeks dostu), 64-bit tamsayı (RAM ve disk tasarrufu). Yöntemler: 1) UUIDv4: 128-bit rastgeledir; zamana göre sıralı değildir, B-Tree indekslerinde sayfa parçalanması (Page Fragmentation) yaratır. 2) UUIDv7: 128-bit'in ilk 48 bitini zaman damgası yaparak sıralı hale getirmiştir. 3) Twitter Snowflake: 64-bit'lik endüstri standardıdır: 1 bit işaret (0), 41 bit zaman damgası (69 yıllık ömür), 10 bit makine/veri merkezi ID'si (1024 bağımsız düğüm), 12 bit sıra numarası (Aynı milisaniyede düğüm başına 4096 ID). Tek bir düğüm saniyede 4 milyon üzerinde sıralı ID üretebilir; sıfır ağ çağrısı, sıfır kilit!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed ID generation requirements: 64-bit integer density, time-ordering (K-sortable to preserve B-Tree append locality), and zero network coordination. 1) UUIDv4: 128-bit completely random; destroys B-Tree index cache efficiency via catastrophic page splits. 2) UUIDv7: Modern 128-bit standard embedding a 48-bit millisecond timestamp prefix for sequential B-Tree friendliness. 3) Twitter Snowflake: The 64-bit gold standard: 1 unused sign bit, 41 bits millisecond epoch timestamp (69 years span), 10 bits machine/datacenter identifier (1024 unique generator nodes), 12 bits sequence counter (4096 IDs per ms per node). A single node generates 4.096 million unique, sortable, collision-free IDs per second in pure local CPU memory without network RPCs.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Gerçek Twitter Snowflake 64-Bit ID Üreteci Implementasyonu
public class SnowflakeIdGenerator {
    private static final long EPOCH = 1704067200000L; // Özel Başlangıç (1 Ocak 2024)
    private static final long WORKER_ID_BITS = 10L;
    private static final long SEQUENCE_BITS = 12L;

    private static final long MAX_WORKER_ID = -1L ^ (-1L << WORKER_ID_BITS); // 1023
    private static final long MAX_SEQUENCE = -1L ^ (-1L << SEQUENCE_BITS);   // 4095

    private final long workerId;
    private long sequence = 0L;
    private long lastTimestamp = -1L;

    public SnowflakeIdGenerator(long workerId) {
        if (workerId > MAX_WORKER_ID || workerId < 0) {
            throw new IllegalArgumentException("Worker ID 0 ile " + MAX_WORKER_ID + " arasında
olmalıdır");
        }
        this.workerId = workerId;
    }

    public synchronized long nextId() {
        long currentTimestamp = System.currentTimeMillis();

        if (currentTimestamp == lastTimestamp) {
            sequence = (sequence + 1) & MAX_SEQUENCE;
            if (sequence == 0) {
                // Aynı milisaniyede 4096 sınırına ulaşıldı -> Sonraki milisaniyeyi bekle!
                while (currentTimestamp <= lastTimestamp) {
                    currentTimestamp = System.currentTimeMillis();
                }
            }
        } else {
            sequence = 0L;
        }

        lastTimestamp = currentTimestamp;

        return ((currentTimestamp - EPOCH) << (WORKER_ID_BITS + SEQUENCE_BITS))
                | (workerId << SEQUENCE_BITS)
                | sequence;
    }
}

void main() {
    var generator = new SnowflakeIdGenerator(7); // Düğüm ID: 7
    for (int i = 0; i < 3; i++) {
        IO.println("Snowflake 64-bit ID: " + generator.nextId());
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Snowflake algoritmasının tek zayıf noktası 'Saat Geriye Kayması' (Clock Moving Backwards) durumudur! Eğer NTP sunucu saatini 5 milisaniye geriye çekerse, daha önce üretilmiş bir ID tekrar üretilebilir (ID Çarpışması!). Gerçek implementasyonlarda saat geri giderse hata fırlatılmalı veya saat yakalanana kadar beklenmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

The critical Achilles' heel of Snowflake is NTP clock skew moving backwards. If system time rolls backward by 5ms, duplicate IDs can collide! Production implementations must throw exceptions or sleep until current time catches up with `lastTimestamp`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Snowflake ID'leri 64-bit boyutuyla veritabanı indekslerinde minimum yer kaplar.
- Doğal zaman sıralaması (K-Sorted) sayesinde en son kayıtlar daima tablonun sonuna eklenir.
- Merkezi bir veritabanı kilit servisine ihtiyaç duymadan yerel olarak üretilir.

🎯 Key Takeaways for the Interview

- 64-bit Snowflake integers maximize B-Tree clustering index density and speed.
- Time-ordered IDs prevent random B-Tree page splits and fragmentation.
- Snowflake nodes operate autonomously without distributed locks or network latency.

35

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık İşlemler: 2-Phase Commit (2PC) vs Saga Pattern (Orchestration vs Choreography)

Distributed Transactions: 2-Phase Commit (2PC) vs Saga Pattern

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. TWO-PHASE COMMIT (2PC) - KATI KİLİT, YAVAŞ, BLOKLAYAN ]
Koordinatör ──1. Hazır mısın? (Prepare)──► [DB 1] (Kilitler ve bekler!)
Koordinatör ──1. Hazır mısın? (Prepare)──► [DB 2] (Kilitler ve bekler!)
Koordinatör ──2. COMMIT / ROLLBACK ──────► İkisi de aynı anda onaylar!

[ 2. SAGA PATTERN (TELAFİ EDİCİ İŞLEMLER / COMPENSATING TRANSACTIONS) ]
Adım 1: Para Çekildi ($100) ──► Başarılı
Adım 2: Stok Rezerve Edildi ──► Başarılı
Adım 3: Kargo Oluşturuldu   ──► HATA ÇIKTI! (Kargo Şirketi Çöktü)
Geri Alma (Compensating):
Adım 2-Ters: Stok Rezervasyonunu İPTAL ET
Adım 1-Ters: Parayı Kullanıcıya İADE ET ($100)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

3 arkadaşın birlikte tatile gitmesini düşünün: 1) İki Fazlı Onay (2PC): 'Üçümüz de aynı anda uçak biletini alıyoruz, 3'e kadar sayacağım: 1-2-3 al!' dersiniz. Biri son anda 'İnternetim koptu' derse diğer ikisi de bileti alamaz, herkes donar kalır! 2) Saga Deseni: Herkes kendi işini sırayla yapar: Ali bileti alır, Veli oteli tutar, Can arabayı kiralar. Can 'Araba kalmamış' derse ne olur? Veli hemen oteli ücretsiz iptal eder, Ali de uçak biletini iade eder (Telafi İşlemi - Compensating Transaction)! Herkes kendi telafi planını çalıştırarak sistemi eski haline döndürür!

🇬🇧 ENGLISH (ELI5)

Booking a vacation across 3 independent systems: 1) Two-Phase Commit (2PC): The coordinator freezes all databases: 'I am counting to 3; everyone lock resources and prepare to commit simultaneously!' If airline booking stalls, hotel and car rental databases sit locked hostage! 2) Saga Pattern: Decoupled local transactions with compensatory rollbacks: Step 1 charges the credit card. Step 2 reserves the flight. Step 3 reserves the hotel. If hotel reservation fails, compensate backwards: Step 2-Compensate refunds the flight, Step 1-Compensate refunds the card! No long-lived database locks!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Two-Phase Commit (2PC / XA): Güçlü ACID garantisi verir. Koordinatör önce 'Prepare' aşamasında tüm düğümlerden 'Hazırım' sözü alır, ardından 'Commit' gönderir. Büyük kusurları: Koordinatör çökerse kilitler sonsuza kadar asılı kalır (Blocking); ağ gecikmeleri nedeniyle throughput berbattır; mikroservislerde ASLA ölçeklenmez. 2) Saga Pattern: Uzun süren dağıtık işlemleri yerel ACID transaction'lar zincirine böler. Bir adım başarısız olursa, önceki başarılı adımlar 'Compensating Transactions' (Telafi Edici İşlemler) ile geri alınır. İki uygulama biçimi: A) Choreography: Olay tabanlıdır (Event-Driven); servisler Kafka olaylarını dinleyip birbirini tetikler. B) Orchestration: Merkezi bir 'Saga Orchestrator' (Devlet Makinesi - State Machine) tüm adımları yönetir ve hata anında telafi adımlarını sırayla tetikler (Büyük sistemlerde takibi kolay olduğu için önerilir).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Two-Phase Commit (2PC): Provides strict linearizable ACID across databases. Coordinator executes Phase 1 (Prepare) where participants acquire locks, then Phase 2 (Commit). Flaws: Synchronously blocks threads and database locks; if the coordinator crashes mid-flight, participants deadlock indefinitely. Strictly unviable for high-scale microservices. 2) Saga Pattern: Decomposes distributed transactions into sequences of isolated local transactions. If a local transaction fails, compensating transactions execute in reverse order to undo side-effects. Execution models: A) Choreography: Decentralized event pub/sub across Kafka topics; difficult to trace at scale. B) Orchestration: A centralized state machine orchestrator coordinates step execution and rollback compensation trees; universally preferred in enterprise architectures for auditability.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Saga Orchestrator (Telafi Edici İşlem Motoru)
import java.util.*;

public class SagaOrchestrator {
    public interface SagaStep {
        boolean execute();
        void compensate(); // Telafi edici geri alma
    }

    public static boolean executeSaga(List<SagaStep> steps) {
        Deque<SagaStep> executedSteps = new ArrayDeque<>();
        for (SagaStep step : steps) {
            if (step.execute()) {
                executedSteps.push(step);
            } else {
                IO.println("⚠ HATA OLUŞTU! Geri sarma (Compensating) başlatılıyor...");
                while (!executedSteps.isEmpty()) {
                    executedSteps.pop().compensate();
                }
                return false;
            }
        }
        return true;
    }
}

void main() {
    var paymentStep = new SagaOrchestrator.SagaStep() {
        public boolean execute() { IO.println("1. Ödeme Alındı: $100"); return true; }
        public void compensate() { IO.println("1-TELAFİ: Kullanıcıya $100 İade Edildi!"); }
    };

    var inventoryStep = new SagaOrchestrator.SagaStep() {
        public boolean execute() { IO.println("2. Stok Rezerve Edilemedi! (HATA)"); return false; }
        public void compensate() { IO.println("2-TELAFİ: Stok Serbest Bırakıldı."); }
    };

    boolean success = SagaOrchestrator.executeSaga(List.of(paymentStep, inventoryStep));
    IO.println("Saga Başarılı mı? " + success);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta mikroservisler arasında veri tutarlılığı sorulduğunda 'Two-Phase Commit (2PC) kurarım' demeyin! 2PC dağıtık sistemlerde performans katili olarak bilinir. Mülakatçıya doğrudan Saga Deseni (Orchestration tabanlı) ve telafi edici işlemleri anlatın.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch 2PC for modern distributed cloud microservices. 2PC causes crippling latency and lock contention. Instantly propose the Saga Pattern with explicit Compensating Transactions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- 2PC kilitleri asılı tuttuğu için mikroservis mimarilerinde ölçeklenemez.
- Saga Deseni işlemi yerel parçalara böler ve hata anında ters telafi adımlarını çalıştırır.
- Orchestrator tabanlı Saga mimarisi, karmaşık akışlarda gözlemlenebilirlik ve yönetim kolaylığı sağlar.

🎯 Key Takeaways for the Interview

- 2PC introduces synchronous lock blocking that cripples microservice throughput.
- The Saga Pattern executes local transactions with reverse compensating rollback hooks.
- Orchestrator-based Sagas provide superior visibility, state tracking, and auditing.

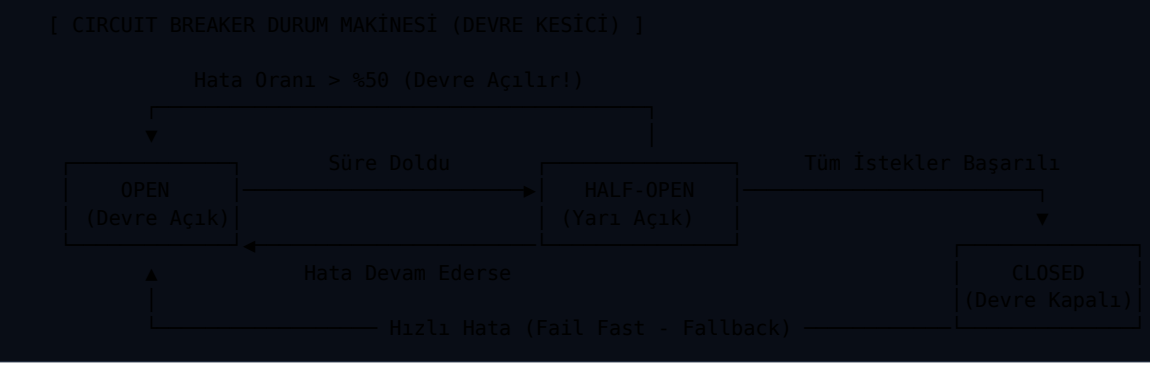
36

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Hata İzolasyonu ve Dayanıklılık: Circuit Breaker, Bulkhead ve Jitter'lı Yeniden Deneme

Fault Tolerance & Resiliency: Circuit Breaker, Bulkhead & Jittered Retry

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1) Circuit Breaker (Evdeki Sigorta): Evde prizden kıvılcım çıkınca tüm binanın yanmasını beklemezsiniz; sigorta 'ÇAT' diye atar (Open). Sorunlu odaya giden elektrik kesilir ama evin geri kalanı yaşamaya devam eder. 2) Bulkhead (Gemideki Su Geçirmez Bölmeler): Titanik gibi devasa bir geminin gövdesi 10 ayrı su geçirmez odaya bölünmüştür. Bir oda delinip su alsa bile su diğer odalara geçmez ve gemi batmaz. 3) Jitter'lı Yeniden Deneme: 100 kişinin aynı kapıyı aynı anda tekmelemesi yerine, herkesin rastgele 1 ile 5 saniye bekleyip kapıyı çalmasıdır!

🇬🇧 ENGLISH (ELI5)

1) Circuit Breaker (Electrical Fuse): When a socket sparks, you don't let the entire house burn down; the fuse trips 'SNAP!' (Open). Electricity isolates that room while the rest of the house stays lit. 2) Bulkhead (Ship Hull Compartments): A submarine hull divides into isolated watertight chambers. If chamber 3 floods, steel doors seal it off; the submarine stays buoyant. 3) Exponential Backoff with Jitter: Instead of 1,000 clients furiously hammering an overwhelmed server every exact 1.0 second (Retry Storm), each client waits an exponentially growing duration plus random jitter (e.g. 1.2s, 2.7s, 5.1s) to disperse the thundering herd!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık dayanıklılığın 3 altın deseni: 1) Circuit Breaker (Resilience4j / Envoy): Kapalı (Normal), Açık (Hata oranı %50'yi aşarsa istekleri uzak servise göndermeden doğrudan Fallback döner - Fail-Fast), Yarı-Açık (Süre dolunca deneme amaçlı birkaç istek geçirir). 2) Bulkhead: Thread havuzlarını ve bağlantı havuzlarını birbirinden izole eder (ör. Ödeme servisi yavaşlasa bile Arama servisinin thread havuzunu tüketip tüm pod'u kilitleyemez). 3) Exponential Backoff with Full Jitter: Uzak servis çöktüğünde yeniden deneme süresini üstel olarak artırmak yetersizdir; tüm istemciler aynı anda denerse 'Retry Storm' oluşur. Formül: Sleep = rand(0, min(Cap, Base × 2^attempt)).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The trifecta of distributed stability: 1) Circuit Breaker: States: Closed (normal execution), Open (failure threshold exceeded, failing-fast immediately with graceful degraded fallback responses), Half-Open (canary testing backend recovery). 2) Bulkhead Pattern: Partitions thread pools, CPU allocations, and database connection pools across domain features. A hung Recommendation engine cannot exhaust the thread pool dedicated to Payment processing. 3) Exponential Backoff with Full Jitter: Simple exponential backoff synchronizes retry spikes into destructive waves (Retry Storms). Amazon AWS proved that injecting randomized Full Jitter (t = random(0, min(M, B × 2^i))) flattens concurrency peaks into smooth traffic profiles.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Exponential Backoff with Full Jitter Implementasyonu
import java.util.concurrent.ThreadLocalRandom;

public class ResilientRetry {
    public static long calculateFullJitterSleep(int attempt, long baseMillis, long capMillis) {
        long exponential = Math.min(capMillis, baseMillis * (1L << attempt));
        return ThreadLocalRandom.current().nextLong(0, exponential + 1);
    }
}

void main() {
    long base = 100; // 100ms
    long cap = 5000; // 5000ms tavan

    IO.println("--- Jitter'lı Yeniden Deneme Bekleme Süreleri ---");
    for (int attempt = 0; attempt < 5; attempt++) {
        long sleep = ResilientRetry.calculateFullJitterSleep(attempt, base, cap);
        IO.println("Deneme #" + (attempt + 1) + " -> Uyku: " + sleep + " ms");
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bağlantı hatası olursa hemen while(true) döngüsüyle tekrar denerim' demeyin! Kontrolsüz yeniden denemeler çökmekte olan bir sunucuyu 'Retry Storm' ile tamamen öldürür. Her zaman Circuit Breaker, Exponential Backoff ve Jitter birlikte kullanılmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch immediate tight-loop retries. Unconstrained retries create catastrophic Retry Storms that obliterate recovering downstream dependencies. Always combine Circuit Breakers with Exponential Backoff and Full Jitter.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Circuit Breaker yavaşlayan servislere giden trafiği keserek hızlı hata (Fail-Fast) üretir.

• Bulkhead deseni kaynak havuzlarını izole ederek tek bir servisin tüm pod'u kilitlemesini önler.

• Full Jitter, yeniden deneme fırtınalarını (Retry Storms) dağıtarak sistemi yumuşakça kurtarır.

🎯 Key Takeaways for the Interview

• Circuit Breakers fail fast during downstream outages, preserving upstream resource capacity.

• The Bulkhead pattern partitions thread pools, isolating failures to single sub-systems.

• Full Jitter stochastically disperses retries, dissipating destructive thundering herds.